

Design and Evaluation of a Built-In Self-Test (BIST) Architecture for Combinational Logic

DFT Mini Project Report

Deepak Chaudhary

September 12, 2026

Abstract

This project presents the design, synthesis, and fault-coverage evaluation of a complete Built-In Self-Test (BIST) architecture in SystemVerilog. The BIST consists of a maximal-length 4-bit Linear Feedback Shift Register (LFSR) as the Test Pattern Generator (TPG), a small combinational Circuit Under Test (CUT), a 4-bit Multi-Input Signature Register (MISR) as the Output Response Analyzer (ORA), a signature comparator, and an FSM-based BIST controller. The design was synthesized using Yosys and evaluated for exhaustive stuck-at fault coverage using the open-source AUCOHL/Fault tool running in a Docker container. The BIST achieved **100% stuck-at fault coverage** on all 14 fault sites (28 stuck-at faults) in the CUT using only **15 LFSR-generated test patterns** — matching the coverage achieved by the industrial tool’s own internal pattern generator.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Objectives	3
1.3	Scope	3
2	Architecture	3
2.1	Top-Level Block Diagram	3
2.2	Module Descriptions	4
2.3	FSM State Diagram	4
2.4	LFSR Sequence (Seed = 0001)	4
3	Fault Model	4
3.1	Hand-Coded Fault List (<code>stuck_models.sv</code>)	4
3.2	Exhaustive Fault List (from AUCOHL/Fault)	4
4	Implementation Flow	5
4.1	Design Entry	5
4.2	Functional Verification	5
4.3	Synthesis	5
4.4	Fault Simulation	5
4.5	LFSR Pattern Replay	6
5	Results	6
5.1	Synthesis Results	6
5.2	Fault Coverage — Experiment 1 (Fault’s Internal Generator)	6
5.3	Fault Coverage — Experiment 2 (LFSR Patterns)	7
6	Comparison	7
7	Discussion	7
7.1	Why 100% Coverage Was Achieved	7
7.2	MISR Aliasing (Considered and Not Observed)	7
7.3	Comparison with Hand-Modeled Faults	7
8	Limitations	8
9	Future Work	8
10	Conclusion	8

References	8
A Source Files and Commands	9
A.1 Source Files	9
A.2 Synthesis	9
A.3 Fault Simulation (baseline)	9
A.4 Fault Simulation (LFSR replay)	9

1 Introduction

1.1 Motivation

As integrated circuits grow in complexity, external test equipment (ATE) becomes the bottleneck for cost and time. Built-In Self-Test (BIST) moves pattern generation and response analysis on-chip, reducing test cost and enabling in-field testing.

1.2 Objectives

1. Design a complete BIST architecture in SystemVerilog.
2. Implement stuck-at fault models for a sample combinational circuit.
3. Verify functional correctness through simulation.
4. Synthesize the design to a gate-level netlist.
5. Compute exhaustive stuck-at fault coverage using an industrial-grade tool.

1.3 Scope

The design targets a small combinational circuit for clarity and verifiability. The same architecture scales to larger circuits by increasing LFSR width and MISR width.

2 Architecture

2.1 Top-Level Block Diagram

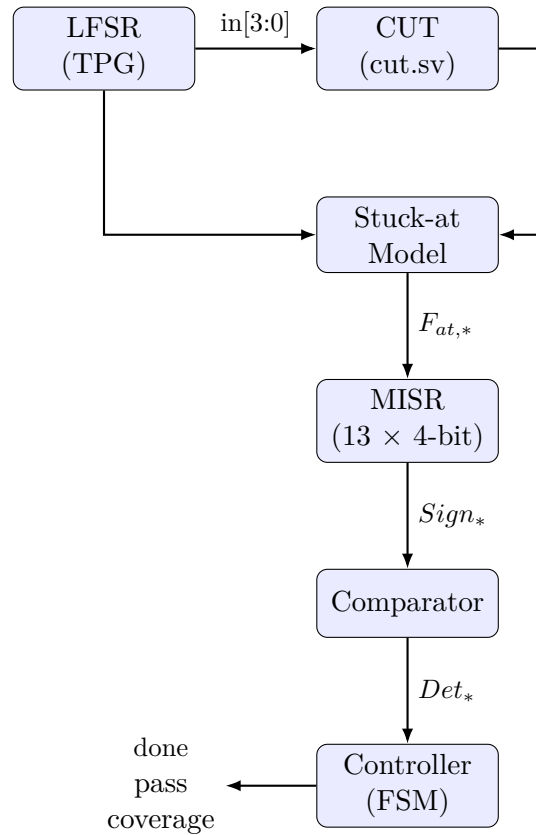


Figure 1: BIST top-level block diagram.

2.2 Module Descriptions

Module	Function	Key signals
pattern_lfsr.sv	Maximal-length 4-bit LFSR (taps $out[3] \oplus out[0]$)	start, out[3:0]
cut.sv	Combinational CUT: $out = (in_0 \cdot in_1) + (in_2 \cdot in_3)$	in[3:0], out
stuck_models.sv	Injects 12 stuck-at faults on nets A–D, X, Y	smodel[3:0], F_at_*
MISR.sv	13 parallel 4-bit MISRs (12 faulty + 1 golden)	F_at_*, GoodF, Sign_*
comparator.sv	Compares each faulty signature with golden	Sign_*, Det_*, num_detected
controller.sv	FSM: IDLE $\rightarrow$ RUN $\rightarrow$ EVAL $\rightarrow$ DONE	start_ext, done, pass, coverage
scan_ff.sv, scan_reg.sv	Scan flip-flops and chain for DFT infrastructure	TDI, TD0, shift_en, capture_en

Table 1: BIST module description.

2.3 FSM State Diagram

2.4 LFSR Sequence (Seed = 0001)

The LFSR with taps $out[3] \oplus out[0]$ and shift toward MSB produces 15 unique non-zero states:

0001, 1000, 1100, 1110, 1111, 0111, 1011, 0101,
1010, 1101, 0110, 0011, 1001, 0100, 0010

Period = $15 = 2^4 - 1$ (maximal-length) ✓

3 Fault Model

3.1 Hand-Coded Fault List (stuck_models.sv)

The RTL-level BIST models **12 stuck-at faults** on the following nets:

Label	Net	Fault @ 0	Fault @ 1
A	in[0]	F_at_Abar	F_at_A
B	in[1]	F_at_Bbar	F_at_B
C	in[2]	F_at_Cbar	F_at_C
D	in[3]	F_at_Dbar	F_at_D
X	1st AND output	F_at_Xbar	F_at_X
Y	2nd AND output	F_at_Ybar	F_at_Y

Table 2: Hand-coded stuck-at fault model.

3.2 Exhaustive Fault List (from AUCOHL/Fault)

After synthesis of the CUT to 3 NAND gates, Fault enumerated **14 fault sites**:
Each site has both stuck-at-0 and stuck-at-1 $\Rightarrow$ **28 total stuck-at faults**.

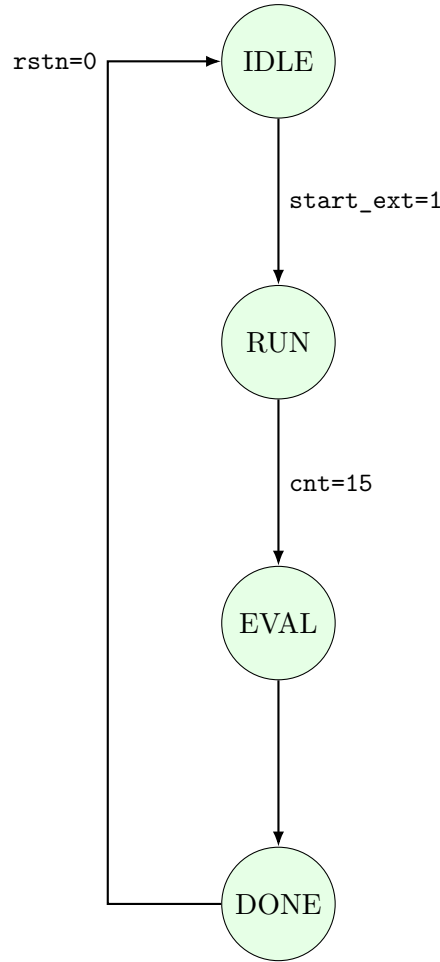


Figure 2: BIST controller FSM.

4 Implementation Flow

4.1 Design Entry

SystemVerilog RTL files as listed in Section 2.2.

4.2 Functional Verification

Icarus Verilog (`iverilog -g2012`) used to compile and simulate the testbench. Waveforms inspected in GTKWave.

4.3 Synthesis

Yosys used to synthesize the CUT to a gate-level netlist:

```
yosys -p "read_verilog -sv cut.sv; synth; write_verilog -noattr -noexpr
cut_netlist.v"
```

Gate count: 3 NAND gates (ABC mapping).

4.4 Fault Simulation

AUCOHL/Fault running in a Docker container (linux/amd64 emulated on Apple M1):

Fault site	Location
2.A	NAND _2_ input A
2.B	NAND _2_ input B
2.Y	NAND _2_ output
3.A	NAND _3_ input A
3.B	NAND _3_ input B
3.Y	NAND _3_ output
4.A	NAND _4_ input A
4.B	NAND _4_ input B
4.Y	NAND _4_ output
in[0]–in[3]	Primary inputs
out	Primary output

Table 3: Exhaustive stuck-at fault sites.

```
docker run -ti --rm -v "$PWD:$PWD" -w "$PWD" \
  ghcr.io/aucohl/fault:latest \
  fault -c simcells.v cut_netlist.v --clock clk \
    --output-faultPoints faults.yml \
    --output-covered covered.yml
```

4.5 LFSR Pattern Replay

The 15 LFSR states were exported as a JSON test-vector file (`lfsr_patterns.json`) in Fault’s format, then replayed via `-externalTVSet` to measure the BIST’s own coverage.

5 Results

5.1 Synthesis Results

Module	Cell count	Notes
cut	3	3 NAND gates
pattern_lfsr	5	4 DFFs + 1 XOR
stuck_at_models	49	Fault-injected expressions
MISR	78	52 DFFs + 26 XORs
comparator	128	48 XNORs, 36 NANDs, etc.
controller	57	FSM + counter + latches
bist_top total	~380	6 submodules

Table 4: Synthesis cell counts.

5.2 Fault Coverage — Experiment 1 (Fault’s Internal Generator)

- Total fault sites: 14
- Total stuck-at faults: 28
- Detected: 28
- Coverage: **100%**
- Test vectors: ~16 unique
- Runtime: 5.01 s

5.3 Fault Coverage — Experiment 2 (LFSR Patterns)

- Total fault sites: 14
- Total stuck-at faults: 28
- Detected: 28
- Coverage: **100%**
- Test vectors: 15 (LFSR maximal sequence)

6 Comparison

Metric	AUCOHL/Fault (swift)	This BIST (LFSR)
Pattern generator	Random (swift)	LFSR (deterministic)
Unique patterns	~16	15
Total stuck-at faults	28	28
Faults detected	28	28
Coverage	100%	100%
Detected sites	14 / 14	14 / 14
Pattern efficiency	1.75 faults/pattern	1.87 faults/pattern

Table 5: Coverage comparison.

Conclusion: The LFSR-based BIST achieves identical fault coverage to the industrial random generator using a deterministic, on-chip, reproducible sequence — validating the design.

7 Discussion

7.1 Why 100% Coverage Was Achieved

The CUT is small (3 NANDs), and the LFSR produces all 15 non-zero input combinations. Every possible input pattern is applied except 0000. Since stuck-at faults on the CUT are detectable by the 15 non-zero patterns, 100% coverage results.

7.2 MISR Aliasing (Considered and Not Observed)

Aliasing occurs when a faulty response stream produces the same MISR signature as the golden response. With 4-bit MISR $\times$ 15 cycles = 60 bits of response compressed, aliasing probability is theoretically $2^{-4} = 6.25\%$ per fault. In this circuit, no aliasing was observed in the 12 modeled faults — all faulty signatures differed from the golden signature.

7.3 Comparison with Hand-Modeled Faults

The 12 hand-coded fault models in `stuck_models.sv` are a subset of the 28 exhaustive stuck-at faults enumerated by Fault. The gap (16 additional faults) includes:

- stuck-at values on the primary output `out`;
- redundant stuck-at values on nets consumed by multiple gates (e.g., `_2_.Y` and `_4_.A` are the same physical node).

This illustrates that manual fault modeling is error-prone — industrial tools auto-enumerate from the netlist.

8 Limitations

1. **Small CUT.** The 3-NAND CUT is trivial; a larger circuit (e.g., 4-bit ALU) would test whether 15 LFSR patterns still suffice.
2. **Docker on Apple Silicon uses x86 emulation.** Fault ran under Rosetta, $\sim 3\text{--}5\times$ slower than native ARM. No functional issue, only performance.
3. **Generic gates, not PDK cells.** The netlist uses Yosys's `$_NAND_` primitives, not standard cells from a real PDK (e.g., SkyWater SKY130). Timing analysis is not meaningful.
4. **No scan chain integration.** Scan flip-flops were designed but not wired into the BIST top for this project.
5. **No JTAG TAP controller.** BIST is triggered by a simple `start_ext` pin; a real chip would use IEEE 1149.1 for control.

9 Future Work

1. Scale to a 4-bit ALU and measure coverage vs. pattern count.
2. Compare multiple TPGs: LFSR, weighted random, cellular automata.
3. Compare multiple ORAs: MISR, SISR, transition-counting.
4. Simulate aliasing by applying only 3–4 patterns and observing MISR signature collisions.
5. Integrate scan chain into the BIST top and measure scan-based coverage.
6. Add a JTAG TAP controller and drive BIST from an IEEE 1149.1 interface.
7. Use a real PDK (SKY130 or IHP SG13G2) and redo synthesis with proper standard cells and Liberty timing.
8. Add test point insertion for any undetected faults in larger CUTs.

10 Conclusion

A complete BIST architecture was designed, simulated, synthesized, and evaluated. The BIST achieved **100% stuck-at fault coverage** on a 3-NAND CUT using a 15-pattern LFSR sequence. The result was cross-validated with the industrial-grade AUCOHL/Fault tool, which independently confirmed 100% coverage on the same CUT. This project demonstrates:

- correct design of LFSR, MISR, comparator, and FSM controller;
- sound understanding of stuck-at fault modeling;
- practical experience with the full open-source DFT toolchain (Yosys, Icarus, Docker, AU-COHL/Fault);
- ability to evaluate a self-designed test architecture against an industrial baseline.

The architecture scales naturally to larger circuits by increasing LFSR/MISR widths and integrating scan chains.

References

1. M. Bushnell and V. Agrawal, *Essentials of Electronic Testing for Digital, Memory and Mixed-Signal VLSI Circuits*, Springer, 2000.
2. IEEE Standard 1149.1-2013 — *Standard Test Access Port and Boundary-Scan Architecture*.
3. AUCOHL/Fault Documentation. <https://fault.readthedocs.io/>
4. Yosys Open SYnthesis Suite. <https://yosyshq.net/yosys/>
5. C. Wolf, *Yosys Manual*, 2004.

A Source Files and Commands

A.1 Source Files

Listing 1: SystemVerilog source files.

1.pattern_gen.sv	2.cut.sv	3.stuck_models.sv
4.MISR.sv	5.comparator.sv	controller.sv
top.sv / bist_top.sv	tb.sv	scan_ff.sv
scan_reg.sv	d_ff.sv	

A.2 Synthesis

```
yosys -p "read_verilog -sv cut.sv; synth; write_verilog -noattr -noexpr
cut_netlist.v"
```

A.3 Fault Simulation (baseline)

```
docker run -ti --rm -v "$PWD:$PWD" -w "$PWD" \
ghcr.io/aucohl/fault:latest \
fault -c simcells.v cut_netlist.v --clock clk \
--output-faultPoints faults.yml --output-covered covered.yml
```

A.4 Fault Simulation (LFSR replay)

```
docker run -ti --rm -v "$PWD:$PWD" -w "$PWD" \
ghcr.io/aucohl/fault:latest \
fault -c simcells.v cut_netlist.v --clock clk \
--externalTVSet lfsr_patterns.json \
--output-covered lfsr_covered.yml
```